

Week 3 Lab / Activity Handout

EDA Design Flow Integration: Understanding FIR Filter Architecture

Item	Details
Week / Date	Week 3 / Thursday, 2026-09-24
Class Length	3 hours
Guided Project	Fixed-point FIR filter (Week 3 uses simple integer tracing; bit-width refinement comes later)
Evaluation Project	Small fixed-point matrix multiplication engine
Course Scoring Reminder	FPGA flow 45%, cell-based digital IC flow 45%, attendance and participation 10%
Submission Type	One PDF file submitted through the course platform

1. Purpose of This Activity

This Week 3 activity begins the guided FIR-filter project. The purpose is not to write a full RTL design yet. The purpose is to understand the architecture clearly before writing Verilog in Week 4.

You will identify the parts of a FIR filter, trace sample movement through delay registers, compute expected outputs by hand, and prepare the signals that should be checked in future waveforms.

Week 3 work supports attendance and participation. It does not directly count toward the 45% FPGA evaluation project or the 45% cell-based IC evaluation project, but it builds the design understanding needed for both.

2. Learning Goals

- Explain what a FIR filter computes using input samples and coefficients.
- Identify taps, delay registers, coefficients, multipliers, adders, and output logic.
- Trace a three-tap FIR example by hand using a small input sequence.
- Separate sequential storage from combinational arithmetic.
- Predict the key waveform signals needed to verify the FIR RTL in Week 4.

3. Activity Schedule for Today

Time	Activity	Expected Result
0-80 min	Teaching slides: FIR concept, taps, delay line, coefficients, multiply-and-add datapath, and hand tracing	You understand how a FIR output is produced from current and previous input samples.
80-90 min	Break	Short pause.
90-155 min	Guided lab activity: Parts A-E	You complete the FIR vocabulary, trace table, classification, and waveform-prediction tasks.
155-165 min	Break	Short pause.
165-180 min	Checkpoint and Week 4 preparation	You confirm what to review before sequential FIR RTL design begins.

4. Software-Only Scope for Week 3

- Week 3 is an architecture-understanding activity, not a board experiment.
- No FPGA board execution is required.
- No ASIC tapeout, fabrication, or silicon validation is involved.
- The evidence for this activity is a completed trace table, architecture reasoning, and short explanations.

5. FIR Architecture Reminder

A finite impulse response (FIR) filter computes an output sample from a finite number of recent input samples. A three-tap FIR can be written as:

$$y[n] = h_0 \cdot x[n] + h_1 \cdot x[n-1] + h_2 \cdot x[n-2]$$

In this equation, $x[n]$ is the current input sample, $x[n-1]$ is the previous input sample, and $x[n-2]$ is the input sample from two cycles earlier. The values h_0 , h_1 , and h_2 are coefficients.

Week 3 convention: d_0 stores $x[n]$ (the newest accepted sample), d_1 stores $x[n-1]$, and d_2 stores $x[n-2]$. During normal operation, at each rising clock edge, the registers update simultaneously using old values: $d_0_new = x_in$, $d_1_new = d_0_old$, $d_2_new = d_1_old$.

In the simple Week 3 architecture, y_out is combinational from d_0 , d_1 , and d_2 . No extra output register and no fixed pipeline-fill delay are assumed. Reset release does not itself shift the delay line; normal sample capture occurs on a later rising clock edge.

6. Part A: Identify FIR Components

Fill in the role of each FIR component using your own words.

Component	Simple role in the FIR filter	Classification in this Week 3 model
Input sample $x[n]$		
Delay register d_0		
Delay register d_1		
Delay register d_2		
Coefficient h_0 , h_1 , h_2		
Multiplier		
Adder network		
Output y_out ($y[n]$)		

7. Part B: Trace a Three-Tap FIR by Hand

Use this example for hand tracing. Assume rst_n has been released before the first capture edge and $d_0 = d_1 = d_2 = 0$ before $x[0]$ is accepted. At each rising edge, update d_0 , d_1 , and d_2 simultaneously from the old state.

Item	Value
Coefficients	$h_0 = 1$, $h_1 = 2$, $h_2 = 1$
Input sequence	$x[0] = 1$, $x[1] = 2$, $x[2] = 3$, $x[3] = 0$, $x[4] = 0$
Initial delayed samples	$x[-1] = 0$, $x[-2] = 0$

The slide shows the first four trace rows as a compact preview; the worksheet adds one final zero sample so you can observe the remaining delayed contribution.

Complete the table. The first row is provided as a guide.

n	x_in	d0 = x[n]	d1 = x[n-1]	d2 = x[n-2]	p0 = h0*d0	p1 = h1*d1	p2 = h2*d2	Expected y_out
0	1	1	0	0	1	0	0	1
1								
2								
3								
4								

8. Part C: Architecture Classification

For each item, classify it as sequential storage/state, combinational arithmetic, a constant, or control. For control signals, note when the answer depends on how that signal is generated in the final design.

Item	Classification	Reason
Delay register d0		
Delay register d1		
Delay register d2		
Coefficient h0, h1, h2		
Products p0, p1, p2		
Adder network and y_out		
Reset behavior of d0, d1, d2		
Optional valid_in / valid_out control signals		

9. Part D: Architecture Description

Write a short description of a three-tap FIR datapath. Mention x_in, delay registers d0/d1/d2, coefficients h0/h1/h2, three multiplier products p0/p1/p2, and an explicit two-adder network: s0 = p0 + p1, then y_out = s0 + p2.

10. Part E: Waveform Prediction for Week 4

Before writing the RTL next week, predict which waveform signals will be useful for proving correct behavior. Do not invent a valid_out latency: include valid_out only if the Week 4 RTL explicitly defines that signal and its latency.

Signal	Why it should appear in the waveform
clk	
rst_n	
x_in	
valid_in (optional; only if defined by the RTL)	

Signal	Why it should appear in the waveform
d0 / delay0	
d1 / delay1	
d2 / delay2	
p0 / p1 / p2	
s0 (partial sum)	
y_out	
valid_out (optional; only if defined)	

11. Evidence Checklist

Evidence item	Required?	Your note
Completed Part A component table	Yes	
Completed Part B FIR trace table	Yes	
Completed Part C classification table	Yes	
Short architecture description from Part D	Yes	
Waveform prediction table from Part E	Yes	
Short exit reflection	Yes	

12. Exit Reflection

Why does a FIR filter need delay registers?

Why are the first few FIR outputs affected by reset history?

Which part of the FIR architecture is most likely to become a timing concern later?

What should you review before Week 4?

13. Preparation for Week 4

Review always @(posedge clk or negedge rst_n), nonblocking assignments, active-low reset behavior, register chains, assign statements, always @(*), and waveform reading. Week 4 will convert the FIR architecture into sequential Verilog RTL and verify it with a testbench.

The accompanying Week 3 source package intentionally provides only a combinational FIR arithmetic reference and testbench-based trace/check examples. It does not provide the full sequential FIR RTL module that students will write in Week 4.